09 — Programmverzweigungen

Entscheidungen im Code mit if, elif, else und match

HTW Berlin – Wirtschaftsingenieurwesen

WiSe 2026/27

Agenda

1. Wozu Verzweigungen?
2. `if` – Code bedingt ausführen
3. Unter der Haube: Kontrollfluss, bool-Weiche, Einrückung
4. `if-else` und `elif` – zwei und mehr Pfade
5. Verzweigung als mathematische Fallunterscheidung
6. `match-case` – strukturierter Mehrfachvergleich
7. **Klassiker: Notenrechner** – live programmiert

Lernziel: Programme treffen Entscheidungen – und Sie **verstehen**, wie der Interpreter aus einer Bedingung genau **einen** Ausführungspfad macht.

Heute: nach jedem Konzept *Live-Demo* (ich) und *Sofort ausprobieren* (Sie, Notebook 09).

Wozu Verzweigungen?

Ohne Verzweigung – das Programm ist starr

```
print("Versand: 4.95 EUR")    # immer gleich
```

Mit Verzweigung – das Programm reagiert

```
if bestellsumme >= 100:
    print("versandfrei")
else:
    print("Versand: 4.95 EUR")
```

Veggie Soles: ab **100 EUR** versandfrei. Verzweigungen lassen Programme **auf Daten reagieren** – erst damit entsteht Geschäftslogik.

`if` – die Wenn-Anweisung

```
bestand = 3
if bestand < 5:
    print("Achtung -- Bestand niedrig!")
```

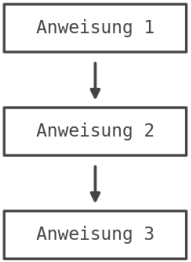
Bestandteile

- `if` – Schlüsselwort
- `bestand < 5` – **Bedingung**, ein Ausdruck aus Notebook 08
- `:` – Doppelpunkt schließt die Bedingung ab
- **Einrückung** (4 Leerzeichen) – markiert den zugehörigen Block

Bedingung **wahr** → Block läuft. Bedingung **falsch** → Block wird übersprungen.

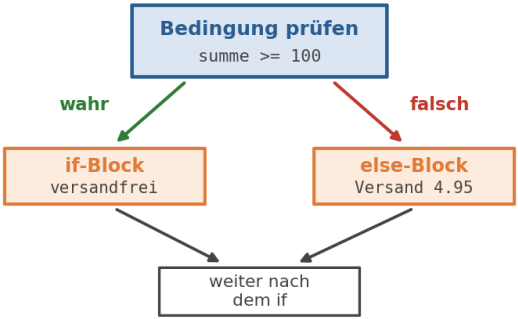
Unter der Haube: der Kontrollfluss verzweigt

bisher: Zeile für Zeile



jede Zeile läuft — immer

mit if/else: genau EIN Pfad



Bisher lief jedes Programm **Zeile für Zeile**. Mit `if` wählt der Interpreter **genau einen Pfad** – der andere Block wird nie ausgeführt. Danach laufen beide Wege wieder zusammen.

Unter der Haube: die Bedingung ist eine bool-Weiche

```
bestellsumme = 89.95
if bestellsumme >= 100:
    print("versandfrei")
else:
    print("Versand: 4.95 EUR")
```

Schritt	Python wertet aus	Ergebnis
1	`bestellsumme >= 100` → `89.95 >= 100`	`False`
2	Pfadwahl anhand des bool-Werts	`else`-Block läuft

Die Vergleichs-Ausdrücke von letzter Woche (NB 08) sind jetzt die **Weichensteller**. Achtung: = weist zu, == vergleicht – `if x = 5:` ist ein SyntaxError. Und `if storniert == True:` ist überflüssig: `storniert` **ist** schon ein bool.

Unter der Haube: Einrückung definiert den Block

Variante A – beides im Block

```
if bestellsumme >= 100:
    print("versandfrei")
    print("Danke!")       # nur bei wahr
```

Variante B – zweite Zeile außerhalb

```
if bestellsumme >= 100:
    print("versandfrei")
print("Danke!")           # läuft IMMER
```

Bei `bestellsumme = 89.95` gibt A **nichts** aus, B gibt `Danke!` aus. Python erkennt Blöcke **nur** an der Einrückung – 4 Leerzeichen, keine Tabs, sonst `IndentationError`.

`if` – Demo & Übung

▶ **Live-Demo** – `01_if_einfach.py`

✎ **Sofort ausprobieren** – Notebook 09, Kap. 2: Temperatur-Warnung, Volljährigkeit, Tagesziel der Produktion

if-else – beide Pfade abdecken

```python
if bestellsumme >= 100:
    print("versandfrei")
else:
    print("Versand: 4.95 EUR")
```

- **Genau einer** der beiden Blöcke läuft – nie beide, nie keiner
- `else` braucht keine eigene Bedingung: “alles Übrige”

▶ **Live-Demo** – `02_if_else_versand.py`

✎ **Sofort ausprobieren** – Notebook 09, Kap. 3: gerade/ungerade, warm/kühl, Passwort-Prüfung

`elif` – mehrere Stufen

```
if bestellsumme >= 200:      # Mengenrabatt
    rabatt = 0.15
elif bestellsumme >= 100:
    rabatt = 0.10
elif bestellsumme >= 50:
    rabatt = 0.05
else:
    rabatt = 0.0
```

- Python prüft **von oben nach unten**, eine Bedingung nach der anderen
- Läuft ein Block, wird der **Rest der Kette übersprungen**
- `else` am Ende ist das **Auffangnetz** für alle übrigen Fälle

Unter der Haube: die elif-Kette ist eine geordnete Prüfung

`bestellsumme = 137.50` – erfüllt `>= 100` **und** `>= 50`:

Prüfung	Bedingung	Ergebnis	Aktion
1	`>= 200`	`False`	weiter zur nächsten
2	`>= 100`	**`True`**	Block läuft: `rabatt = 0.10`
3	`>= 50`	wird **nicht mehr geprüft**	–

Die **erste wahre Bedingung gewinnt** – obwohl 137.50 auch `>= 50` ist, bekommt die Bestellung 10 %, nicht 5 %. Genau deshalb ist die **Reihenfolge** Teil der Logik.

Die Reihenfolge-Falle – Demo & Übung

```
if bestellsumme >= 50:        # zu allgemein zuerst!
    rabatt = 0.05
elif bestellsumme >= 100:     # unerreichbar
    rabatt = 0.10
```

Jede Summe ≥ 50 bleibt in Stufe 1 hängen – die 10 % werden **nie** vergeben. Regel: **höchste/spezifischste Stufe zuerst**, absteigend prüfen.

▶ **Live-Demo** – `03_elif_rabattstufen.py`

✎ **Sofort ausprobieren** – Notebook 09, Kap. 4: Notensystem, Temperaturklassen, Fehlerquote

Formal: Verzweigung = Fallunterscheidung

```
if s >= 200:    rabatt = 0.15
elif s >= 100:  rabatt = 0.10
elif s >= 50:   rabatt = 0.05
else:           rabatt = 0.0
```

$$\text{rabatt}(s) = \begin{cases} 0.15 & s \geq 200 \\ 0.10 & 100 \leq s < 200 \\ 0.05 & 50 \leq s < 100 \\ 0 & s < 50 \end{cases}$$

Die Mathematik verlangt **disjunkte** Fälle. Die elif-Kette schafft das **durch die Reihenfolge** – deshalb genügt im Code `s >= 100`.

Bedingungen kombinieren – and, or, not

```
if stammkunde and bestellsumme >= 100:
    print("VIP-Versand")
elif bestellsumme >= 100 or premium:
    print("Standard-Versand kostenlos")
else:
    print("Versand 4.95 EUR")
```

- Die Logik-Operatoren aus Notebook 08 leben hier auf
- Klammern erhöhen Lesbarkeit – besonders bei and/or-Mix

▶ **Live-Demo** – `04_kombinierte_bedingungen.py`

✎ **Sofort ausprobieren** – Notebook 09, Trainingsmaterial Aufg. 6: Wetter + Temperatur kombinieren

match-case – strukturierter Mehrfachvergleich

```
match modell:
    case "Eco-Sneaker":
        print("recyceltes Polyester")
    case "Hemp-High":
        print("Obermaterial aus Hanf")
    case _:
        print("nicht im Sortiment")
```

- `match` nimmt **einen** Wert und vergleicht ihn Fall für Fall
- `case _:` ist der **Default-Fall** (wie `else`)
- Mehrere Werte pro Fall: `case "rot" | "gelb":`
- Verfügbar ab **Python 3.10**

Unter der Haube: `match` vs. `elif`-Kette

Auch `match` prüft **von oben nach unten**, der erste Treffer gewinnt – aber jeder `case` ist ein fester **Gleichheits-Vergleich** mit demselben Wert:

Wann besser `match`?	Wann besser `if`/`elif`?
viele exakte Werte (`"A"`, `"B"`, …)	Bereiche (`x >= 50`, `x < 18`)
eine Variable, viele Fälle	mehrere Variablen kombiniert
die Variable steht nur **einmal** da	komplexe `and`/`or`-Bedingungen

Faustregel: *“ist gleich diesem konkreten Wert”* → `match`. *“ist größer/kleiner als”* → `if`/`elif`.

`match-case` – Demo & Übung

▶ **Live-Demo** – `05_match_case.py`

✎ **Sofort ausprobieren** – Notebook 09, Kap. 6: Jahreszeiten aus Monat, Note als Text, Ampelsteuerung

Klassiker: Notenrechner – die Aufgabe

- **Punkte** (0–100) einlesen – Achtung: `input()` liefert einen String
- Die **Note** nach dem HTW-Raster ermitteln und ausgeben
- Unter 50 Punkten gibt es die **5.0** – das Auffangnetz

So soll es laufen

```
$ python3 notenrechner.py
Punkte: 87
Note: 1.7

$ python3 notenrechner.py
Punkte: 42
Note: 5.0
```

Klassiker: Notenrechner – das Raster

ab Punkten	Note	ab Punkten	Note
95	1.0	70	2.7
90	1.3	65	3.0
85	1.7	60	3.3
80	2.0	55	3.7
75	2.3	50	4.0

darunter: **5.0** – im Code eine **absteigende** elif-Kette:

```
if punkte >= 95:      note = 1.0
elif punkte >= 90:    note = 1.3
# ... absteigend weiter bis >= 50 ...
else:                 note = 5.0
```

Klassiker: Notenrechner – Live-Coding

▶ **Live-Coding ab leerem Editor** – Referenz: `90_klassiker_notenrechner.py`
Teilziele:

1. Punkte einlesen: `int(input("Punkte: "))`
2. Grenzfall zuerst: höchste Stufe `>= 95` → `1.0`
3. elif-Kette **absteigend** bis `>= 50`
4. `else` als Auffangnetz → `5.0`
5. Reihenfolge-Falle prüfen: was passiert bei **aufsteigender** Kette?

✎ **Zum Selberlösen** – Handout *"Klassiker 09: Notenrechner"* (`Klassiker/09_Klassiker_Notenrechner.pdf`). Vertiefung: Notebook 09, Kap. 4 + Trainingsmaterial.

Cheat Card

Aufgabe	Syntax
eine Bedingung	`if cond:`
zwei Pfade	`if cond: ... else: ...`
mehrere Stufen	`if c1: ... elif c2: ... else: ...`
Bedingungen kombinieren	`if a and (b or c):`
exakte Werte	`match x: case "a": ... case _: ...`
bool direkt nutzen	`if storniert:` statt `== True`
Klassiker heute	Notenrechner: elif-Kette absteigend + `else`

Faustregel: Bereiche → `if`/`elif`, von der spezifischsten zur allgemeinsten Stufe. Exakte Werte → `match`. Und immer fragen: *gewinnt wirklich die richtige Bedingung zuerst?*

Ausblick: Notebook 10 – Schleifen

Unser Notenrechner bewertet **eine** Klausur – dann ist das Programm zu Ende. 30 Klausuren? 30-mal neu starten…

```
for punkte in klausur_punkte:    # Vorschau NB 10
    ...                          # Raster anwenden
```

Nächste Woche: **Schleifen** (`for`, `while`) – die Regel einmal schreiben, der Rechner wiederholt sie beliebig oft. Die Syntax lernen wir dann – heute nur der Appetit.

Heute geübt

- ✓ `if` – Code bedingt ausführen, Doppelpunkt + Einrückung
- ✓ Unter der Haube: der Interpreter wählt **genau einen** Pfad
- ✓ Die Bedingung ist eine bool-Weiche (Ausdrücke aus NB 08)
- ✓ Einrückung definiert den Block – was gehört zum `if`?
- ✓ `elif`: geordnete Prüfung, die **erste wahre** Bedingung gewinnt
- ✓ Verzweigung = mathematische Fallunterscheidung
- ✓ `match-case` für exakte Werte, `case _:` als Default
- ✓ **Klassiker Notenrechner** live gesehen

✎ **Ihre Aufgaben bis zur nächsten Woche:**

- Klassiker **Notenrechner** eigenständig lösen (Handout, inkl. Bonus)
- “Sofort ausprobieren”-Aufgaben im Notebook 09
- Trainingsmaterial: BMI-Bewertung, Rabatt-Programm, Schichtplanung